

AchiVAI — AI Resume Enhancer: Complete Technical Report

Repository: Z:\AI\ai_resume (also published as Aman17123/AchiVAI-AI-SaaS_RESUME-ENHANCER) **Author of code:** Aman Nakoti **Date of analysis:** August 13, 2026

Table of Contents

- 1. Executive Summary
- 2. Tech Stack & Architecture
- 3. Project Structure
- 4. File-by-File Breakdown
- 5. Entry Points & Execution Flow
- 6. Interconnectivity Map
- 7. Key Workflows Traced End-to-End
- 8. Data Flow & State Management
- 9. Configuration & Environment
- 10. Dependencies Explained
- 11. Code Quality Observations
- 12. Glossary of Terms

1. Executive Summary

AchiVAI is a full-stack, AI-powered SaaS web application ("AI Resume Enhancer") that helps job seekers get past Applicant Tracking Systems (ATS). It is the kind of software that recruiters' software scans before a human ever sees a resume — so the product's core value proposition is: *upload your resume, get an ATS compatibility score, see which keywords are missing, and rebuild it in an ATS-safe template.*

The product solves two aligned problems:

1. **Analysis problem** — Candidates have no objective idea how their resume performs against the automated scanners that reject ~75% of applications. AchiVAI uploads and parses a PDF/DOCX, optionally compares it with a pasted job description, and asks Google Gemini (model `gemini-3.6-flash`) to return a structured JSON verdict: an ATS score out of 100, matched vs. missing keywords, per-section feedback, and five actionable suggestions.
2. **Rebuild problem** — Fixing a resume requires a clean, parser-friendly format. AchiVAI provides an interactive resume builder with two templates (Classic and Modern), a live preview, localStorage autosave, cloud save to a per-user account, and a one-click print-ready PDF export.

The audience is primarily **students, freshers, and professionals in India** (pricing is in ₹, payments go through Razorpay, payment methods are UPI/cards/net banking). The business model is freemium: a free plan allows 2 AI analyses per month, while Premium is a one-time ₹499 upgrade that unlocks unlimited analyses. Deployment is on Vercel at `arcivai.vercel.app`.

In short: this is a **monolithic Next.js application with server-rendered pages, client-side React interactivity, three external service integrations (Supabase, Google Gemini, Razorpay)**, and a moderate amount of structure — roughly 60 source files across 15 directories. No tests exist; code quality is generally good but contains notable dead code (detailed in Section 11).

2. Tech Stack & Architecture

2.1 Stack

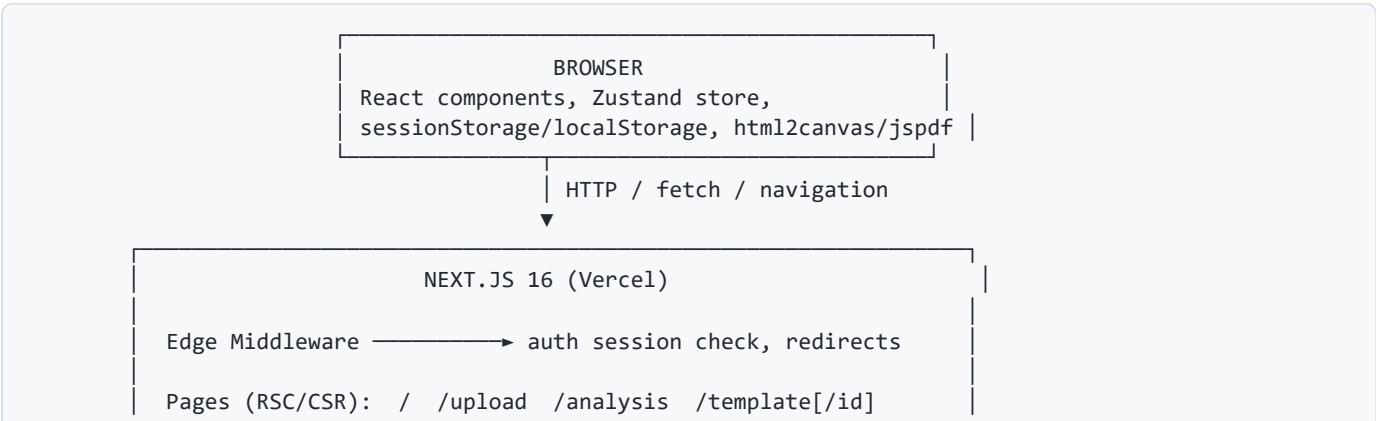
Layer	Technology	Version	Role in the app
Framework	Next.js (App Router)	16.3	Server rendering, routing, API route handlers, middleware, metadata/SEO
UI library	React	19.2.0	Client components for interactivity
Styling	Tailwind CSS v4 + @tailwindcss/postcss	4.x	Utility-first styling; PostCSS pipeline
Animation	Framer Motion	12.23	Scroll-reveal animations on marketing pages
Icons	Lucide React	0.552	Icon set throughout UI
Global state	Zustand	5.0.8	Resume data store shared between editor form and preview
Database & Auth	Supabase (PostgreSQL + GoTrue)	supabase-js 2.112	Auth (OAuth/email/magic link), tables with RLS
AI	Google Generative AI (@google/generative-ai)	0.24	Resume analysis (Gemini 3.6 Flash)
Payments	Razorpay	2.9.8	Order creation, signature verification server-side; JS SDK client-side
PDF parsing	pdf-parse (+ pdfjs-dist 5.4.x), mammoth	2.4.5 / 1.12	Extract text from PDF and DOCX uploads
PDF export	jspdf + html2canvas	4.2 / 1.4	Client-side screenshot-to-PDF export
Misc libs	clsx , tailwind-merge , country-state-city	—	Class merging; country/state/city autocomplete data
Language	JavaScript (JSX), with TypeScript limited to next-env.d.ts , 3 SEO files and config	—	The whole app is JS; TS is only used where Next.js generates it

2.2 Architecture pattern

This is a **server-rendered monolith with an API layer** — a standard modern Next.js App Router architecture. There is no separate backend service; everything runs in the single Next.js deploy (Vercel serverless functions). The three "services" (Supabase, Gemini, Razorpay) are all **external**, accessed over HTTPS. Key characteristics:

- **Server Components / Client Components split:** Marketing pages and static things like layout metadata render on the server; pages with interactivity (`page.jsx` files marked `"use client"`) hydrate in the browser.
- **Route Handlers as the API:** `src/app/api/**/route.js` files are the server-side API (analysis, payment, plan queries).
- **Edge middleware** (`src/middleware.js`) runs before routes to check the Supabase session cookie and redirect unauthenticated users.
- **Three Supabase client flavors:** a browser client, a server client using SSR cookies, and an RLS-bypassing admin client (service role key) used only inside payment route handlers.

2.3 Architecture diagram (text/ASCII)



/dashboard /login /pricing

Route Handlers (API)

POST /api/analyze-resume → Google Gemini 3.6
POST /api/razorpay/create-order → Razorpay API
POST /api/razorpay/verify |
POST /api/razorpay/webhook ← Razorpay webhooks
GET /api/user/plan

Server clients: supabaseServer (SSR cookies), supabaseAdmin



SUPABASE (hosted PostgreSQL + Auth + RLS)
tables: profiles, resumes, subscriptions,
usage_events

3. Project Structure

```
ai_resume/
├── .env.local                # Local secrets & config (GITIGNORED)
├── .gitignore               # Excludes node_modules, .next, .env*, keys
├── README.md                # Full product documentation (features, setup, schema)
├── package.json              # Scripts + 18 runtime dependencies
├── package-lock.json         # Lockfile
├── next.config.mjs           # Image domains whitelist, server packages for pdf-parse
├── postcss.config.mjs       # Tailwind v4 PostCSS plugin
├── eslint.config.mjs         # ESLint flat config (next core-web-vitals)
├── jsconfig.json             # JS path alias: @/* → src/*
├── tsconfig.json             # TS config (allowJs), Next-managed
├── next-env.d.ts             # Next.js generated TS declarations
├── scripts/
│   └── supabase.sql          # One-shot DB schema: tables, triggers, RLS policies
├── public/
│   ├── logo.svg, icon.svg    # Brand assets
│   ├── images/               # bg.png, cursor.png, brand logos (google/amazon/nvidia/tiktok)
│   └── template-previews/     # classic.png, modern.png preview cards
├── src/
│   ├── middleware.js         # Edge auth guard for protected routes
│   ├── app/                  # Next.js App Router (routes = folders)
│   │   ├── layout.js         # Root layout: fonts, global SEO metadata, JSON-LD schema
│   │   ├── page.jsx          # Homepage ("/"): composes all marketing sections
│   │   ├── globals.css       # Tailwind v4 entry + custom CSS
│   │   ├── icon.svg          # Favicon
│   │   ├── error.js / loading.js / not-found.js # App-wide error, loading, 404 screens
│   │   ├── robots.ts / sitemap.ts / opengraph-image.tsx # SEO machine files
│   │   ├── _component/       # 13 reusable marketing/UI components
│   │   │   ├── NavBar.jsx, Hero.jsx, Footer.jsx, Mid.jsx ...
│   │   │   ├── HowItWorksSection, ResumeHighlight, HeroResumeSection,
│   │   │   ├── WhyAchiVAISection, PlansSection, TestimonialsSection,
│   │   │   └── HoverResume, FAQSection
│   │   ├── upload/           # Resume upload + job description → analysis
│   │   ├── analysis/         # ATS report display page
│   │   ├── template/ [id]/    # Template gallery + resume editor (form + live preview)
│   │   ├── dashboard/        # User's saved resume library
│   │   ├── login/            # Auth page (Google, email, magic link, reset)
│   │   ├── pricing/          # Plans + Razorpay checkout trigger
│   │   ├── auth/callback/route.js # OAuth code→session exchange handler
│   │   └── api/
│   │       ├── analyze-resume/route.js # Core AI analysis (POST, multipart)
│   │       ├── razorpay/      # create-order, verify, webhook handlers
│   │       └── user/plan/route.js # Current plan lookup (GET)
│   ├── components/
│   │   ├── Editor/ResumeForm.jsx # Data-driven form (sections, fields, autocomplete)
│   │   ├── PDFTemplates/PDFTemplateFactory.jsx # Template lookup for PDF export
│   │   └── ResumeRenderers/      # ClassicTemplate.jsx, ModernTemplate.jsx (preview)
│   ├── data/universities.js      # University name list for autocomplete
│   ├── lib/
│   │   ├── supabase.js          # Browser Supabase client factory
│   │   ├── supabaseServer.js    # Server client (SSR cookie-backed)
│   │   ├── supabaseAdmin.js     # Service-role client (RLS bypass; server-only)
│   │   ├── fileParsers.js       # PDF/DOCX → plain text extraction
│   │   ├── prompt.js            # Gemini prompt builder
│   │   ├── resumeService.js     # Cloud save of resumes
│   │   ├── razorpayServer.js    # Razorpay client + HMAC verifiers + price/limits
│   │   └── utils.js             # cn() class-merge helper
│   ├── store/resumeStore.js      # Zustand global resume data store
│   ├── templates/
│   │   └── templateSchemas.js   # (unused) static section lists
│   └── utils/downloadPDF.js      # html2canvas + jsPDF export pipeline
```

The structure follows Next.js conventions exactly: **folders under** `src/app` **become URLs**, `route.js` files become API endpoints, and layout files wrap child pages with shared metadata. The separation of `lib/` (server + client helpers) from `components/` (UI) from `store/` (state) is clean and idiomatic.

4. File-by-File Breakdown

4.1 Root configuration files

File	Purpose	Key contents	Imports / is imported by
<code>package.json</code>	Dependency and script manifest	Scripts: <code>dev</code> , <code>build</code> (both with <code>--webpack</code>), <code>start</code> , <code>lint</code> (<code>eslint</code>). 18 runtime deps	N/A (consumed by npm/Next)
<code>next.config.mjs</code>	Next.js build config	<code>images.remotePatterns</code> whitelists <code>i.pimg.com</code> , <code>images.unsplash.com</code> , <code>cdn.pixabay.com</code> ; AVIF/WebP formats; <code>minimumCacheTTL: 86400</code> ; <code>serverExternalPackages: ["pdf-parse", "pdfjs-dist"]</code> so native/node deps of the PDF parser are excluded from bundling	N/A
<code>postcss.config.mjs</code>	Tailwind v4 processing	Loads <code>@tailwindcss/postcss</code>	N/A
<code>eslint.config.mjs</code>	Flat ESLint config	Uses <code>eslint-config-next/core-web-vitals</code> , ignores <code>.next/</code> , <code>out/</code> , <code>build/</code>	N/A
<code>jsconfig.json</code>	Path aliasing for JS files	<code>"@/*": ["*"]</code> with <code>baseUrl: "src"</code> — imports such as <code>@/lib/...</code> resolve to <code>src/lib/...</code>	Consumed by editor tooling
<code>tsconfig.json</code>	TS compiler config	<code>allowJs: true</code> , <code>strict: false</code> , bundles <code>next-env.d.ts</code> and <code>.next</code> type files	Generated types consumed by IDE
<code>next-env.d.ts</code>	Next.js generated	Ambient type shims for JSX/Next internals	N/A
<code>.gitignore</code>	VCS exclusions	<code>node_modules/</code> , <code>.next/</code> , <code>.env*</code> (except <code>.env.example</code>), <code>*.pem</code> , etc.	N/A
<code>.env.local</code>	Local secrets (gitignored)	Supabase URL + anon key, service role key, Gemini key, Razorpay key/secret, <code>NEXT_PUBLIC_APP_URL</code>	Read by every lib that reads <code>process.env</code>

4.2 `scripts/supabase.sql`

The **complete database schema**, meant to be pasted into the Supabase SQL Editor once. It defines:

- `profiles` — one row per user (`id` FK → `auth.users`), synced automatically by a trigger.
- `resumes` — the core document table: `user_id` (FK), `name`, `template` (string: `classic` / `modern`), `data` (JSONB — the entire resume object), unique `share_token`, timestamps.
- `subscriptions` — per-user plan row: `plan` (`free` / `premium`), `status` (`active` / `inactive`), `razorpay_subscription_id`, `current_period_end`. Unique on `user_id`.
- `usage_events` — append-only log rows (`user_id`, `kind` = `'analysis'`, `created_at`) used to count the free monthly quota.
- `handle_new_user()` trigger function — on every new `auth.users` insert it creates both a `profiles` row and a `subscriptions` row (plan defaults to `free`).
- RLS policies** — every table has Row Level Security; a user can only select/insert/update/delete their own rows (`auth.uid() = user_id`).
- `get_resume_by_token(token)` — a `security definer` SQL function to fetch resume JSON by share token. **Note: no application route calls this yet** (public share links are unfinished).

4.3 `src/middleware.js` — edge auth guard

The only middleware file. Runs on **every request** except static assets (matcher regex excludes `_next/static`, images, favicons, template previews, and common image extensions).

- Creates a Supabase SSR client over the request cookies and calls `supabase.auth.getUser()`.

- `PROTECTED_ROUTES = ["/template/", "/dashboard", "/analysis"]` — redirects unauthenticated users to `/login?next=<original path>` (so they return where they were headed after sign-in).
- If a logged-in user visits `/login`, redirects them to `/dashboard`.
- `PUBLIC_ROUTES` is declared but **never used** (dead code).

Imports: `@supabase/ssr`, `next/server`. **Imported by:** nobody (framework invokes it).

4.4 `src/lib/` — business/plumbing layer

`supabase.js` (browser client factory)

- `createClient()` → `createBrowserClient(url, anonKey)` from `@supabase/ssr`. Returns a Supabase client that manages its own token storage in cookies.
- **Imported by:** `Navbar`, `login/page`, `dashboard/page`, `resumeService`, `template/[id]/page`, `PlansSection`.

`supabaseServer.js` (server client factory)

- `createServerSupabaseClient()` — async; reads the `next/headers` cookie store and wraps it in Supabase SSR's `getAll / setAll` cookie adapters. This is how server components and route handlers share the user's session.
- **Imported by:** `api/analyze-resume`, `api/user/plan`, `api/razorpay/create-order`, `api/razorpay/verify`.

`supabaseAdmin.js` (privileged client)

- `getSupabaseAdmin()` — lazily creates a client with `SUPABASE_SERVICE_ROLE_KEY` and `persistSession: false`; throws if the key is missing. Service-role client **bypasses RLS**, so it is only used by server code that must write someone else's rows (payment activation). A module-level `cachedClient` avoids recreating the client per call.
- **Imported by:** `api/razorpay/verify`, `api/razorpay/webhook`.

`fileParsers.js` (upload text extraction)

- `extractText(file, mimeType)` — takes a browser `File` and its MIME type; returns extracted plain text (capped at `MAX_CHARS = 30000`).
- PDF path: uses `pdf-parse` v2's `PDFParse` class loaded via `createRequire` (CommonJS interop under ESM), with the `pdfjs-dist` worker resolved explicitly to silence fake-worker warnings. DOCX path: `mammoth.extractRawText({ buffer })`.
- Post-processing: collapses all whitespace runs to single spaces (`/\s+/g → " "`), strips page markers like `-- 3 of 5 -`, and throws a clear error for empty/scanned-image PDFs.
- **Imports:** `node:module` (`createRequire`), `mammoth`, `pdf-parse`, `pdfjs-dist`. **Imported by:** `api/analyze-resume`.

`prompt.js` (AI prompt builder)

- `buildAnalysisPrompt({ resumeText, jobDescription })` — a pure function returning a single string. It instructs Gemini to act as an "expert ATS resume reviewer", optionally includes the JD, requires **strict JSON only** with an exact schema (score, keyword arrays, `sectionsAnalyzed` object, feedback, 5 suggestions, `suggestedRole`), and encodes scoring rules (penalize missing contact info, weak verbs, no quantified achievements).
- **Imports:** none. **Imported by:** `api/analyze-resume`.

`razorpayServer.js` (payments plumbing)

- Constants: `PREMIUM_PRICE_PAISE = 49900` (₹499) and `FREE_MONTHLY_LIMIT = 2`.
- `getRazorpay()` — constructs the server SDK client from `RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET`.
- `verifyRazorpaySignature(orderId, paymentId, signature)` — recomputes the HMAC-SHA256 of `orderId|paymentId` with the key secret and compares hex digests.
- `verifyWebhookSignature(body, signature)` — same technique over the raw request body, preferring `RAZORPAY_WEBHOOK_SECRET` with fallback to the key secret.
- **Imports:** `razorpay`, `node:crypto`. **Imported by:** all three `api/razorpay/*` routes and `api/analyze-resume` (for `FREE_MONTHLY_LIMIT`).

`resumeService.js` (cloud persistence)

- `saveResumeToCloud({ name, template, data })` — resolves the current user via the browser client; if signed in, **upserts** the `resumes` row. It remembers the last saved resume id in `sessionStorage["achivai_resume_id"]`: if present, it updates that row; otherwise it inserts and stores the new id. Returns a status object `{ ok, auth, message }` (the `auth` flag is true when a login was required; the caller interprets `ok:false, auth:false` as "saved offline").

- **Imports:** `./supabase` . **Imported by:** `template/[id]/page` .

`utils.js` (UI helper)

- `cn(...inputs)` — `twMerge(clsx(inputs))` for conflict-free Tailwind class merging. **Never imported anywhere** (dead code; the codebase inlines class strings).

4.5 `src/store/resumeStore.js` — global state

Zustand store named `useResumeStore` . It owns the **entire resume document** as one object under `data` :

Action	Behavior
<code>setFullData(newData)</code>	Replaces <code>data</code> with a deep clone (safe against mutation aliasing)
<code>updateField(field, value)</code>	Shallow-updates a top-level field (<code>name</code> , <code>email</code> , ...)
<code>updateArrayField(field, index, key, value)</code>	Mutates a deep clone of the array item <code>data[field][index][key]</code>
<code>addArrayItem(field, emptyObj)</code>	Pushes a new item (auto-.items get an <code>id</code> assigned by the form)
<code>removeArrayItem(field, index)</code>	Splices out an item
<code>moveArrayItem(field, fromIndex, toIndex)</code>	Reorders (guarded bounds) — defined but never called (the form implements its own move logic)
<code>snapshot()</code>	Returns a deep clone of current data — never called
<code>saveResume(resumeData)</code>	Writes the whole object to <code>localStorage["resume_saved"]</code>

Imports: `zustand` . **Imported by:** `ResumeForm` , `template/[id]/page` .

4.6 `src/utils/downloadPDF.js` — PDF export engine

`downloadResumePDF(data, theme, templateLayout)` — the single most intricate client-side routine. Steps:

1. Picks the template component via `getPDFTemplate(templateLayout)` .
2. Creates a **hidden off-screen A4-width container** (`div style="width:210mm; position:fixed; left:-10000px"`).
3. Server-renders the template to HTML with `ReactDOMServer.renderToString` and injects it.
4. Waits 800 ms for fonts to load.
5. Finds "section boundary" markers — elements with the CSS class `pdf-section-start` — and records their absolute Y offsets (this is how pagination avoids slicing through a heading).
6. Screenshots the container with `html2canvas` (scale 2, `useCORS` , taint allowed) while an `onclone` hook normalizes exotic CSS colors (`lab()` , `oklch()` , `color()`) to plain `rgb()` because `html2canvas` cannot parse them (they would otherwise render black).
7. Computes page slices: A4 aspect ratio `297/210` , fills each page to 95% (`MAX_FILL`), snapping to the nearest section boundary; hard cap of 60 pages.
8. Creates a `jsPDF` instance (A4 portrait) and paints the single tall canvas onto each page with a negative Y offset (`pdf.addImage(imgData, "PNG", 0, -topMm, imgWidth, imgHeight)`).
9. Saves the file as `<name>_<templateLayout>.pdf` .

Inputs: resume `data` , `theme` (ignored internally), `templateLayout` string. **Output:** boolean success + browser download.

Imports: `html2canvas` , `jspdf` , `react` , `react-dom/server` , `PDFTemplateFactory` . **Imported by:** `template/[id]/page.jsx` .

4.7 `src/components/` — UI building blocks

`ResumeRenderers/ClassicTemplate.jsx` (client component)

- Pure presentational: given `data` , renders a one-column serif resume: centered name, contact line, then EDUCATION → SKILLS → PROJECTS → EXPERIENCE. It normalizes several skill shapes (string, `{value, description}` , `{skillName, description}`) so legacy data formats still render. Each `SectionHeader` carries `className="pdf-section-start"` , which is the pagination hook for PDF export. Very compact typography (8–15 px).
- **Imports:** `react` . **Imported by:** `PDFTemplateFactory` , `template/[id]/page` .

ResumeRenderers/ModernTemplate.jsx (client component)

- Two-column layout (33% sidebar / 67% body). Sidebar holds a photo placeholder, name/title, summary paragraphs (handles string or array), contact, "references" note. Body holds WORK EXPERIENCE, EDUCATION, PROJECTS, SKILLS (chips), LANGUAGES, CERTIFICATIONS — each section header also tagged `pdf-section-start`. Emoji icons (📁 🎓 🚀 🔧 🧠 🏆) appear in the circled icons. Sections that are empty simply vanish (missing entries degrade gracefully to "No experience added." messages).
- **Imports:** `react`. **Imported by:** `PDFTemplateFactory`, `template/[id]/page`.

PDFTemplates/PDFTemplateFactory.jsx

- `getPDFTemplate(templateType)` — switch returning the component (`modern` → `ModernTemplate`, defaults to `ClassicTemplate`).
- `availableTemplates` — array of `{id, name, component}` — **exported but never imported** (dead code; the editor hard-codes template options).
- **Imports:** both renderers. **Imported by:** `downloadPDF.js`.

Editor/ResumeForm.jsx (client component, 869 lines — the biggest file) A **configuration-driven** form. Instead of hand-written fields it declares:

- `TEMPLATE_SECTIONS` — which resume sections each template allows (classic: 7 sections incl. languages/certifications; modern: 5).
- `SECTION_CONFIG` — for each section: `isArray`, `defaultItem`, `fields` list, and per-field metadata (type fallback, `autocomplete` source, textarea `multiline`, grid `span`).
- `FIELD_PROPS` — per-field validation rules (email regex, phone pattern, CGPA range 0–10, url/date types).
- `ICONS` — lucide icon per field name.

Core behavior:

- Subscribes to the Zustand store; every keystroke writes straight into `store.data`, so the live preview updates in real time.
- **Autosave:** debounced (900 ms) write of the whole document to `localStorage["resume_autosave_v_final"]`; on mount, it reloads that key into the store if the store is empty (crash recovery).
- **ID fix pass:** any array item missing an `id` gets `crypto.randomUUID()` so React keys and reorder logic stay stable.
- **Autocomplete fields:** `InputField` debounces queries (300 ms) and filters from static sources — countries/states/cities from `country-state-city`, universities from `src/data/universities.js`, degrees and fields-of-study from local lists. City resolution cascades: given country+state it lists that state's cities; without a state it unions the first 5 states' cities.
- `ArraySection` renders each item as a card with up/down/delete buttons; `handleMove` implements reordering *in the form itself* (not via the store's `moveArrayItem`).
- To avoid hydration mismatch it renders a skeleton (`PolishedSkeleton`) until mounted.
- Exports `getCountryList`, `getStateListForCountry`, `getCityListForState` (used by no other file — only internal).

Imports: `react`, `../../store/resumeStore`, `country-state-city`, `lucide-react`, `../../data/universities`.

Imported by: `template/[id]/page.jsx`.

4.8 src/app/ — routes, pages, layouts

layout.js (RootLayout, server component) — the application shell.

- Loads three Google fonts via `next/font/google`: Geist Sans, Geist Mono, Josefin Sans (brand font).
- Exports an extensive `metadata` object: title template, description, keywords, authors, canonical URL, OpenGraph, Twitter card, icons, robots.
- Injects **JSON-LD structured data** (Schema.org): `SoftwareApplication`, `Organization`, `WebSite`, and `FAQPage` (4 FAQs) for SEO rich results.
- `viewport` theme color `#021F81`.
- Wraps children in `<html lang="en">` with `suppressHydrationWarning`.

page.jsx (homepage, client) — thin composition: `Navbar` + `Hero` and then **dynamically imported** (code-split) marketing sections: `HowItWorksSection`, `ResumeHighlight` (ssr:false — it uses motion + mouse position), `HeroResumeSection`, `Mid` (upload CTA), `WhyAchiVAISection`, `PlansSection`, `TestimonialsSection`, `HoverResume`, `FAQSection`, `Footer`.

upload/page.jsx — the analysis entry point.

- Client-side file validation: MIME whitelist (`application/pdf` , DOCX, `application/msword` for legacy .doc) and 5 MB max — mirroring the server checks.
- On "Analyze with AI": builds `FormData` , POSTs to `/api/analyze-resume` , handles 401 by redirecting to `/login?next=/upload` , then stores the JSON result in `sessionStorage["achivai_result"]` (chosen over URL params to avoid huge URLs) and routes to `/analysis` .
- Fetches `/api/user/plan` on mount to show (or hide) the "Free plan: 2 analyses/month" banner.

analysis/page.jsx — the report renderer.

- Reads `sessionStorage["achivai_result"]` (guarded try/catch); if absent, redirects to `/upload` .
- `ScoreRing` component: an SVG progress ring colored by score thresholds (≥ 80 green "Excellent", ≥ 60 amber "Needs Work", else red "Weak").
- Renders matched/missing keyword chips, a numbered improvement list, section-by-section feedback cards, suggested role, and a CTA to rebuild with a template.

template/page.jsx — template gallery. Hero + scrolling gallery of two `ResumeCard` s (preview PNGs from `public/template-previews/`); clicking routes to `/template/<id>` .

template/[id]/page.jsx — the **resume editor** (620 lines, the second biggest file).

- Resolves `templateId` from the URL: `new` → blank document; `?resume=<id>` → loads the saved JSONB straight from the `resumes` table via the browser client (and transfers it from another account's edit to the current session's `sessionStorage["achivai_resume_id"]`); otherwise loads `classic.json` / `modern.json` as seed data.
- Maintains UI state: `previewMode` (desktop/mobile), `zoomLevel` (50–150%), `isFullscreen` , `activeSection` , notifications, export/save flags.
- Layout: top action bar (Save → cloud, Export → PDF), left sidebar section menu (+ "Add Custom Section"), center form (`<ResumeForm activeSection=...>`), right **live preview** rendering the selected `TemplateComponent` with the shared Zustand `data` .
- `handleSave` → `saveResumeToCloud(...)` ; on `ok:false`, `auth:false` it falls back to local `saveResume(data)` and shows "Saved offline...".
- `handleExportPDF` → `downloadResumePDF(data, theme, templateLayout)` (note: `theme` is UI-state but does nothing today).
- Template picker modal switches `templateLayout` (`classic` / `modern`) at runtime, which swaps the preview component — a neat "change skin without losing data" trick, though the underlying seed data shapes differ slightly.
- `handleShare` copies the current URL (only shares the `editor` link, not a public view).

dashboard/page.jsx — user's resume library. On mount creates the browser client, verifies the user (redirect to `/login` otherwise), and lists `resumes` ordered by `updated_at desc` . Cards show name, template chip, last-updated date; actions: **Edit** (`/template/<template>?resume=<id>`), **Delete** (`window.confirm` then `supabase.from('resumes').delete()`), **Sign out**, plus a "New Resume" link.

login/page.jsx — auth UI with four providers: Google OAuth (`signInWithOAuth` → redirect to `/auth/callback?next=...`), email/password (login or signup mode), magic link (`signInWithOtp`), and password reset (`resetPasswordForEmail`). All resolver callbacks point at `${NEXT_PUBLIC_APP_URL}/auth/callback` . Wrapped in `Suspense` because it reads `useSearchParams` .

pricing/page.jsx — plans plus the payment choreography:

- Loads `/api/user/plan` to know `guest|free|premium` .
- `handlePurchase` : guest users get a "sign in" error; otherwise POST `/api/razorpay/create-order` , lazy-loads the Razorpay checkout script (`https://checkout.razorpay.com/v1/checkout.js`), opens the checkout modal with the returned order, and in the success `handler` POSTs to `/api/razorpay/verify` with order/payment/signature. On verification success the UI switches to "You're on Premium".

auth/callback/route.js — OAuth/magic-link callback endpoint.

- Reads `?code` and `?next` from the URL, creates a Supabase SSR client bound to the **response** cookie jar, calls `exchangeCodeForSession(code)` , then redirects to `next` (default `/dashboard`). On error redirects to `/login?error=...` . Contains leftover `console.log` debugging calls that print the full incoming URL (including the one-time code).

`api/analyze-resume/route.js` — the AI endpoint (see Workflow 1, Section 7.1 for full walkthrough). Enforces auth, premium check or free quota count, file validation, extraction, Gemini call with `temperature: 0.4` and `responseMimeType: "application/json"`, resilient JSON parsing (direct `JSON.parse`, falling back to regex-extracting the first `{...}` block), normalization (`normalize()` clamps score 0–100, coerces arrays), and finally records a `usage_events` row for non-premium users (quota counts post-analysis, so failed calls do not consume quota).

- Response shape `{atsScore, matchedKeywords, missingKeywords, sectionsAnalyzed, feedback, suggestions, suggestedRole, jobDescription, plan}`.

`api/razorpay/create-order/route.js` — signed-in check; asks Razorpay for `orders.create({ amount: 49900, currency: "INR", receipt: "<userId>-<ts>", notes: { userId } })`; returns `{orderId, amount, currency, keyId}`.

`api/razorpay/verify/route.js` — 4-step verify: (1) user must be logged in; (2) HMAC signature must match; (3) **server-side double check** that the payment is actually `captured` (`payments.fetch`); (4) `getSupabaseAdmin().from('subscriptions').upsert({...plan:'premium', status:'active', razorpay_subscription_id}, {onConflict:'user_id'})` — the admin client is required here because a normal user client cannot write others/own subscription rows that were created by the signup trigger (RLS only permits `select` on subscriptions).

`api/razorpay/webhook/route.js` — Razorpay server-to-server callback. Verifies the HMAC over the raw body using `x-razorpay-signature`, then switches on event type: `payment.captured` / `order.paid` → upsert premium via admin client; `payment.failed` / `payment.refunded` → set `inactive`. User attribution relies on `payment.notes.userId` (or the order-level notes) that was stamped at order creation. A missing `userId` is treated as "already received" (no-op) rather than an error.

`api/user/plan/route.js` — GET; returns `{plan: 'guest'|'free'|'premium', email, user_id}`. `premium` only when `plan='premium' AND status='active'`. Gracefully returns `guest` on any error.

`error.js` / `not-found.js` / `loading.js` — App Router conventions: a global error boundary with a "Try again" reset button, a branded 404 page, and a generic spinner.

`robots.ts` / `sitemap.ts` / `opengraph-image.tsx` — SEO machine files: robots allows all + sitemap URL; sitemap lists 6 routes (note: includes private routes `/dashboard` and `/analysis`); OG image generated at runtime via `ImageResponse` (1200×630, brand navy background).

`_component/Navbar.jsx` — fixed top nav; listens to `scroll` to toggle a blurred background; subscribes to Supabase `onAuthStateChange` so the CTA flips between "Login / Sign up" and "Dashboard" in real time. Accent colors are passed as props by pages (white-on-dark for `upload`, navy for `analysis` / `pricing`).

`_component/Hero.jsx`, `HeroResumeSection.jsx`, `ResumeHighlight.jsx`, `HoverResume.jsx`, `Mid.jsx` — pure marketing sections. Hero: headline + CTA cards using **remote stock images** from `i.pinimg.com` (hence the `remotePatterns` config). ResumeHighlight: custom cursor + spring-animated flip/zoom card choreography with Framer Motion. HoverResume: three static CV images with hover lift. Mid: upload/template call-to-action buttons.

`_component/HowItWorksSection.jsx` / `WhyAchiVAISection.jsx` / `PlansSection.jsx` / `TestimonialsSection.jsx` / `FAQSection.jsx` — content sections (3-step explainer, 6 feature cards, pricing cards with a premium handler that routes to `/login?next=/pricing` when signed out, 3 fake testimonials, and an accordion FAQ). All are `"use client"` for Framer Motion `whileInView`.

4.9 `src/data/universities.js`

- `UNIVERSITIES` — a static array of ~25 well-known university names (Harvard → IIT Delhi → Tsinghua...). Only used for education autocomplete in `ResumeForm`.

4.10 `src/templates/`

- `classic.json` / `modern.json` — **seed form data** with empty shapes. Classic uses `{title, company, ...}`, `{institution, degree, school, year, description}`, `{skillName, description}`, plus `keyAchievements` (never rendered). Modern uses `{role, company}`, `{name, tech}`, `{degree, school, year}`. These two files use **different key names** than the form's `defaultItem` schemas (`ResumeForm` uses `experience.title`, editors use `{company, title}`), which causes some mismatched empty fields to persist — the renderers tolerate this by normalizing display shapes.
- `templateSchemas.js` — exports per-template section lists; **imported nowhere** (dead code).

4.11 public/

Static assets: `logo.svg`, `images/bg.png` (login background), `images/cursor.png` (custom magnifier cursor), `images/logos/*.svg` (Google/Amazon/Nvidia/TikTok brandmarks), `template-previews/{classic,modern}.png` (gallery cards). All served as-is by Next.

5. Entry Points & Execution Flow

5.1 Boot sequence (per page view)

1. Request hits Vercel edge (or `next dev` server)
2. `src/middleware.js` runs FIRST for every non-static route
 - builds Supabase SSR client from cookies → `auth.getUser()`
 - unauthenticated + protected route? → redirect `/login?next=...`
 - authenticated + `/login?` → redirect `/dashboard`
3. `src/app/layout.js` is the ROOT layout for all routes
 - loads Google fonts (Geist / Geist Mono / Josefin Sans)
 - emits global `<metadata>` (SEO, OG, Twitter) in `<head>`
 - injects 4 JSON-LD schema.org `<script>` blocks
 - renders `{children}` inside `<html><body>`
4. Route segment:
 - a. Layout files for the folder (`login/template/upload/...` export extra metadata like page `<title>`)
 - b. ``page.jsx`` for that segment renders (client components hydrate)
5. Global ``loading.js`` (spinner) shows during Suspense transitions;
``error.js`` catches render/runtime errors; ``not-found.js`` for 404s.

5.2 Route map

URL	Page file	Guard
<code>/</code>	<code>app/page.jsx</code>	public
<code>/upload</code>	<code>app/upload/page.jsx</code>	public (analysis call requires auth)
<code>/analysis</code>	<code>app/analysis/page.jsx</code>	auth required (middleware) ; data from sessionStorage
<code>/template</code>	<code>app/template/page.jsx</code>	public
<code>/template/:id</code>	<code>app/template/[id]/page.jsx</code>	auth required (middleware)
<code>/dashboard</code>	<code>app/dashboard/page.jsx</code>	auth required (middleware)
<code>/login</code>	<code>app/login/page.jsx</code>	public (reverse-guard → dashboard)
<code>/pricing</code>	<code>app/pricing/page.jsx</code>	public (purchase requires sign-in at click-time)
<code>/auth/callback</code>	<code>app/auth/callback/route.js</code>	public
POST <code>/api/analyze-resume</code>	<code>api/analyze-resume/route.js</code>	auth + quota
POST <code>/api/razorpay/*</code>	three route files	auth + signature
GET <code>/api/user/plan</code>	<code>api/user/plan/route.js</code>	public (guest-safe)

5.3 Order of initialization inside the editor (`/template/:id`)

1. Page mounts in "loading" state (`ready=false`).
2. `useEffect(load): new ? → makeEmpty(); ?resume= ? → Supabase select; else template JSON seed → setFullData(...) → ready=true`.
3. `ResumeForm` mounts → **mount guard** (`mounted` state, skeleton until overlay) → then: restore `resume_autosave_v_final` from localStorage if store empty; start autosave watcher; ensure array `id` s exist.
4. Every keystroke → store update → preview re-renders (same `data`), autosave timer (900 ms) armed.
5. "Save" → cloud upsert (+ localStorage fallback). "Export" → PDF pipeline.

6. Interconnectivity Map

Textual dependency graph — an arrow `A → B` means "A imports/calls B".

6.1 Static import graph (module level)

```
middleware.js → @supabase/ssr, next/server
app/layout.js → next/font/google, ./globals.css
app/page.jsx → _component/{Navbar, Hero, HowItWorks, ResumeHighlight,
               HeroResume, Mid, WhyAchiVAI, Plans, Testimonials,
               HoverResume, FAQ, Footer} (dynamic)
app/upload/page.jsx → _component/{Navbar, Footer}, framer-motion
app/analysis/page.jsx → _component/Navbar, lucide-react
app/pricing/page.jsx → _component/Navbar, framer-motion
app/login/page.jsx → lib/supabase (createClient), public/images/bg.png
app/dashboard/page.jsx → lib/supabase (createClient)
app/template/page.jsx → _component/Navbar, framer-motion
app/template/[id]/page.jsx → components/Editor/ResumeForm
                           ▶ components/ResumeRenderers/{Classic,Modern}
                           ▶ store/resumeStore (useResumeStore)
                           ▶ templates/{classic,modern}.json
                           ▶ utils/downloadPDF
                           ▶ lib/{resumeService, supabase}

components/Editor/ResumeForm → store/resumeStore, data/universities,
                               country-state-city, lucide-react
components/PDFTemplates/Factory → components/ResumeRenderers/{Classic,Modern}
utils/downloadPDF → components/PDFTemplates/Factory,
                   react-dom/server, html2canvas, jsPDF
lib/fileParsers → mammoth, pdf-parse, pdfjs-dist
lib/resumeService → lib/supabase
lib/razorpayServer → razorpay, node:crypto
lib/supabaseServer → @supabase/ssr, next/headers
lib/supabaseAdmin → @supabase/supabase-js

api/analyze-resume → lib/{fileParsers, prompt, supabaseServer, razorpayServer}
api/razorpay/create-order → lib/{supabaseServer, razorpayServer}
api/razorpay/verify → lib/{razorpayServer, supabaseAdmin, supabaseServer}
api/razorpay/webhook → lib/{razorpayServer, supabaseAdmin}
api/user/plan → lib/supabaseServer
auth/callback → @supabase/ssr
```

6.2 Runtime message flow (HTTP + storage)

```
Browser client (Zustand, sessionStorage, localStorage)
| fetch POST /api/analyze-resume (multipart: file + jobDescription)
| | ↳ Supabase auth check → usage_events count → extractText
| |   ↳ Gemini JSON → normalize → response
| | sessionStorage["achivai_result"] → /analysis page
|
| fetch POST /api/razorpay/create-order → Razorpay order
| Razorpay checkout (client SDK) → handler → POST /api/razorpay/verify
|   ↳ verify signature → payments.fetch(captured?) → admin upsert subscriptions
| Razorpay server webhook → POST /api/razorpay/webhook → admin upsert
|
| GET /api/user/plan → subscriptions row → {plan, email}
|
| Supabase browser client (direct, RLS-protected):
|   resumes: select/insert/update/delete (dashboard, editor save/load)
|   auth:   getUser, signInWithOAuth, signInWithPassword, signUp,
|           signInWithOtp, resetPasswordForEmail, signOut,
|           onAuthStateChange (Navbar)
```



```
localStorage: "resume_autosave_v_final" (form autosave),
              "resume_saved" (manual save fallback)
sessionStorage: "achivai_resume_id" (cloud save id),
                "achivai_result" (analysis report)
```

6.3 Shared-component topology

The **Zustand store is the hub** of the editor: `ResumeForm` (writes) and the preview `TemplateComponent` (reads) both subscribe to the same `data`, so no prop-drilling is needed. The PDF exporter *does not* read the store — it receives a `data` snapshot argument at call time. Supabase clients are created per-module (factory functions), so there is no shared singleton except the lazily-cached admin client.

7. Key Workflows Traced End-to-End

7.1 Workflow A — AI Resume Analysis (the core loop)

Trigger: User lands on `/upload`, picks a `.pdf/.docx` file, optionally pastes a job description, clicks "Analyze with AI".

- `upload/page.jsx` validates MIME + size client-side (`VALID_TYPES`, `MAX_SIZE = 5MB`), then `fetch("/api/analyze-resume", { method: "POST", body: formData })` — a multipart form with `file` and optional `jobDescription`. A 401 response redirects to `/login?next=/upload` (line 70–78).
- `api/analyze-resume/route.js` (Node runtime, `force-dynamic`):
 - Env guard:** returns 500 if `GEMINI_API_KEY` or Supabase URL/anon key are missing (lines 19–32).
 - Auth:** `createServerSupabaseClient()` → `supabase.auth.getUser()`; 401 if anonymous (lines 35–45).
 - Quota gate:** queries `subscriptions` (RLS `select` allowed). If not premium, counts `usage_events` rows with `kind='analysis'` and `created_at >= startOfMonth` (first day of month at 00:00); if `>= FREE_MONTHLY_LIMIT` (2) → 402 with an upgrade hint (lines 47–76).
 - File gate:** re-validates type and size on the server; then `extractText(file, file.type)` (lines 84–96).
 - AI call:** builds the prompt via `buildAnalysisPrompt(...)` and calls `genAI.getGenerativeModel({ model: "gemini-3.6-flash", generationConfig: { responseType: "application/json", temperature: 0.4 } })` → `result.response.text()` (lines 98–111).
 - Parsing resilience:** `JSON.parse(raw)`; on failure, regex-extracts `/\{[\s\S]*\}/` and re-parses; requires `atsScore` to be numeric or it 500s (lines 113–126).
 - Quota write-back:** non-premium users get one `usage_events` insert (so the next request sees the decremented allowance) (lines 129–133).
 - Response:** `{...normalize(analysis), jobDescription, plan}` where `normalize` clamps/rounds `atsScore` and defends every array/string field (lines 149–159).
- `upload/page.jsx` stores the JSON in `sessionStorage["achivai_result"]` (deliberately not a URL param — comment says "to avoid huge URLs") and calls `router.push("/analysis")` (lines 84–85).
- `analysis/page.jsx` hydrates from the same key on mount (lazy `useState` initializer with try/catch), redirects to `/upload` if null, and renders: `ScoreRing` gauge (SVG stroke-dasharray animation, color by band), suggested role, overall `feedback`, green/red keyword chips, numbered `suggestions`, and section cards from `sectionsAnalyzed`. A "Rebuild with an AchivAI template" CTA links to `/template`.

Inputs → Outputs: PDF/DOCX bytes + optional JD text → on-screen ATS report + one `usage_events` row per free analysis.

7.2 Workflow B — Build a Resume and Export PDF

Trigger: User clicks a template card on `/template` (navigates to `/template/classic` or `/template/modern`).

- `template/page.jsx` — gallery cards are clickable `motion.div`s doing `router.push( /template/${id} )`.
- `app/template/[id]/page.jsx` mount effect:
 - `templateId === "new"` → blank document (empty arrays, `_template: "new"`).
 - `?resume=<id>` → `supabase.from("resumes").select(...).eq("id", resumeId).maybeSingle()`; if found, `setFullData(saved.data)` — the cloud JSONB becomes the current document (lines 88–110).
 - else → seed from `templates/classic.json` or `modern.json` (lines 112–128); `templateLayout` is derived from the id containing "modern".
- Form ↔ store ↔ preview loop:** `ResumeForm` renders the active section's fields from `SECTION_CONFIG`; every change calls a store action → `data` updates → the right-hand `<ClassicTemplate data={data}/>` or `<ModernTemplate>` re-renders instantly. Array reorders and adds happen in the form; the store's `deepClone` prevents accidental aliasing.
- Autosave:** any change schedules `localStorage["resume_autosave_v_final"]` write after 900 ms (line 382–389). Reload restores it if the store is empty — crash recovery.
- Cloud save** (`handleSave`): `saveResumeToCloud({name, template, data})`. Signed out → `{ok:false, auth:true}` prompts sign-in; signed in → insert (first time, stores id to `sessionStorage["achivai_resume_id"]`) or update that row by id + `user_id` guard (lines 33–50 of `resumeService.js`). Busy flag shows "Saving...".

6. **PDF export** (`handleExportPDF` → `downloadResumePDF`): the hidden A4 container → `renderToString(<TemplateComponent data={data}/>)` → 800 ms font wait → collect `.pdf-section-start` Y-offsets → `html2canvas` at 2× with exotic-color normalization → A4 slice math (pages snap to section tops, 95% fill, 60-page cap) → `jsPDF.addImage` with negative offsets → `pdf.save("<name>_<layout>.pdf")` . On any throw it cleans up the DOM node and returns false (lines 97–101 of `downloadPDF.js`).
7. **Dashboard round-trip**: the saved row appears on `/dashboard` ; "Edit resume" re-opens `/template/<template>?resume=<id>` ; "Delete" removes the row via the browser client (RLS-scoped to own rows).

7.3 Workflow C — Premium Purchase (Razorpay)

Trigger: User on `/pricing` clicks "Go Premium" (or the hero's PlansSection button which redirects here via `/login?next=/pricing` if signed out).

1. **pricing/page.jsx** on mount calls `GET /api/user/plan` → `{plan: "guest" | "free" | "premium"}` . Premium users see "You're on Premium" instead of the button.
2. **Order creation:** `handlePurchase` → `POST /api/razorpay/create-order` → server requires a session (401 otherwise) → `getRazorpay().orders.create({amount: 49900, currency: "INR", receipt: "<userId>-<epoch>", notes: {userId}})` → returns `orderId/amount/currency/keyId` (the public Razorpay key travels safely to the client; the secret never does).
3. **Checkout:** client lazily injects `checkout.razorpay.com/v1/checkout.js` , then `new window.Razorpay({... handler})` , and `rzp.open()` . The browser handles UPI/card/netbanking inside the hosted modal; `payment.failed` fires a UI error.
4. **On success handler** → `POST /api/razorpay/verify` with `{orderId, paymentId, signature}` :
 - 1. session required → 2) `verifyRazorpaySignature` HMAC-SHA256 over `orderId|paymentId` → 3) `razorpay.payments.fetch(paymentId)` must be `captured` (defense against forgery) → 4) admin client `upsert` into `subscriptions` (`plan: "premium", status: "active", onConflict: "user_id"`).
 - Response `{success: true, plan: "premium"}` → UI banner "Payment successful!"
5. **Server-side fallback (webhook):** Razorpay independently POSTs `payment.captured` / `order.paid` (or failure/refund events) to `/api/razorpay/webhook` with an `x-razorpay-signature` header. The route verifies the HMAC over the **raw body** (never the parsed JSON), extracts `userId` from the payment/order notes, and admin-upserts/updates `subscriptions` accordingly. This guarantees premium activation even if the user closes the browser mid-flow.
6. **Effect on the quota gate:** the next `/api/analyze-resume` call sees `plan='premium' + status='active'` and skips the usage count entirely — unlimited analyses.

8. Data Flow & State Management

8.1 Layers of data movement

Layer	Mechanic	What lives there
Browser state	Zustand <code>useResumeStore.data</code>	The one authoritative resume document while editing
Browser persistence	<code>localStorage</code>	<code>resume_autosave_v_final</code> (debounced autosave), <code>resume_saved</code> (manual backup)
Tab/session persistence	<code>sessionStorage</code>	<code>achivai_result</code> (analysis report, survives <code>router.push</code>), <code>achivai_resume_id</code> (current cloud row id)
Server-session	Supabase Auth cookies (<code>sb-*</code> managed by <code>@supabase/ssr</code>)	JWT session; read by middleware, server clients, API routes
Database	Supabase Postgres tables with RLS	<code>resumes</code> (JSONB documents), <code>subscriptions</code> , <code>usage_events</code> , <code>profiles</code>
External APIs	HTTPS	Gemini (AI), Razorpay (orders/payments/webhooks)

8.2 The document lifecycle

1. **Seed** — `templates/*.json` (or `makeEmpty`, or a cloud row) → `store.setFullData(deepClone)` .
2. **Edit** — form actions replace the object immutably (data flows store→form and store→preview).
3. **Persist** — autosave → `localStorage`; Save → Supabase `resumes` (upsert semantics via session-stored id).
4. **Export** — snapshot passed by value to `downloadResumePDF` (no store dependency → avoids subscription churn during the heavy screenshot work).
5. **Analyze** — file text and JD go **server-side only**; the result travels back through the HTTP response and then `sessionStorage` .

8.3 Auth state distribution

The session cookie is the single source of truth. `Navbar` additionally subscribes to `onAuthStateChange` for instant UI updates; `dashboard` and the editor re-check `getUser()` on mount; API routes re-verify the session on **every** call (defense in depth); middleware guards route access at the edge.

8.4 Quota accounting (state machine)

```
subscribe row? plan=premium & status=active → bypass quota
no ↓
count usage_events(kind='analysis', month-start..now)
  < 2 → allow, then INSERT one row (fire-and-forget)
  ≥ 2 → 402 { "Free plan limit reached..." }
```

Premium activation flips the branch by upserting the `subscriptions` row (verify route or webhook). Note the write happens *after* a successful analysis, so failed calls never burn quota — but a concurrent burst could exceed 2 before the counters catch up (no row-level locking).

8.5 No caching layer

There is no Redis/memcached/CDN cache usage in application code; only Next's default static/dynamic rendering and the image cache TTL (`minimumCacheTTL: 86400`). The AI endpoint is `force-dynamic` .

9. Configuration & Environment

9.1 Environment variables (from `.env.local` ; all must exist in Vercel too)

Variable	Scope	Controls
<code>NEXT_PUBLIC_SUPABASE_URL</code>	public	Base URL for all three Supabase clients
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	public	Non-privileged client key (RLS applies)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	server-only	Admin client (RLS bypass) — used in verify/webhook
<code>GEMINI_API_KEY</code>	server-only	<code>analyze-resume</code> AI calls
<code>RAZORPAY_KEY_ID</code>	public (via API response) + server	Checkout modal identity + order creation
<code>RAZORPAY_KEY_SECRET</code>	server-only	HMAC signature verification
<code>RAZORPAY_WEBHOOK_SECRET</code>	server-only	Webhook HMAC (falls back to key secret)
<code>NEXT_PUBLIC_APP_URL</code>	public	Auth redirect URLs, metadata base, sitemap/robots

9.2 Build-time config

- `next.config.mjs` — image remote whitelist (pimg/unsplash/pixabay), image formats + TTL, `serverExternalPackages` for `pdf-parse` / `pdfjs-dist`.
- `postcss.config.mjs` — Tailwind v4.
- `eslint.config.mjs` — `next/core-web-vitals`.
- `jsconfig.json` — `@/*` → `src/*`.
- `tsconfig.json` — `allowJs`, JSX react-jsx, includes `.next` types.
- `README.md` documents all of the above for local setup and Vercel deployment.

9.3 Hard-coded business config (no env)

- `PREMIUM_PRICE_PAISE = 49900` and `FREE_MONTHLY_LIMIT = 2` live in `lib/razorpayServer.js`.
- File limits (`5MB` , MIME whitelist) duplicated client (`upload/page`) and server (`analyze-resume`) — intentional defense-in-depth, though a single shared constant would be cleaner.
- Model name `"gemini-3.6-flash"` and `temperature: 0.4` hard-coded in the route.

10. Dependencies Explained

Package	Version	Used for	Where
next + react / react-dom	16.3 / 19.2	App Router, SSR, routing, route handlers, middleware; <code>renderToString</code> for PDF HTML	everywhere
@supabase/ssr	0.12	Cookie-managed session clients (browser + server + middleware)	lib/supabase* , middleware , auth/callback
@supabase/supabase-js	2.112	Admin client (service role)	lib/supabaseAdmin
@google/generative-ai	0.24	Gemini 3.6 Flash structured JSON analysis	api/analyze-resume
razorpay	2.9	Server SDK: orders, payment fetch	lib/razorpayServer , API routes
pdf-parse (v2)	2.4	PDF text extraction (<code>PDFParse</code> class, pdf.js backed)	lib/fileParsers
pdfjs-dist	5.4	PDF worker required by <code>pdf-parse</code> (declared as <code>serverExternalPackage</code>)	transitive, <code>fileParsers</code>
mammoth	1.12	DOCX → raw text extraction	lib/fileParsers
jspdf	4.2	A4 PDF assembly from canvas slices	utils/downloadPDF
html2canvas	1.4	DOM screenshot of the rendered template	utils/downloadPDF
zustand	5.0	Global resume document state	store/resumeStore , editor
framer-motion	12.23	Scroll/hover animations across marketing UI	_component/* , upload, template gallery
lucide-react	0.552	Icon library	most components
tailwind-merge + clsx	3.3 / 2.1	Class merging (<code>cn</code>) — <i>currently unused in production code</i>	lib/utils (dead)
country-state-city	3.2	Country/state/city autocomplete datasets	components/Editor/ResumeForm
Dev: eslint / eslint-config-next , @tailwindcss/postcss , tailwindcss , @types/*	—	Linting, Tailwind v4 pipeline, type stubs	build tooling

11. Code Quality Observations

11.1 Dead or unused code

Item	File	Evidence	Suggested action
<code>templateSchemas.js</code>	<code>src/templates/</code>	Never imported (grep confirms zero references)	Delete, or wire into <code>ResumeForm</code> as the source of truth
<code>cn()</code>	<code>src/lib/utils.js</code>	Never imported anywhere	Delete, or start using for conditional classes
<code>moveArrayItem</code> + <code>snapshot()</code>	<code>src/store/resumeStore.js</code>	Never called (form reimplements move; snapshot unused)	Remove or use in editor for undo history
<code>availableTemplates</code>	<code>PDFTemplateFactory.jsx</code>	Never imported (editor hard-codes <code>templateOptions</code>)	Remove or use in gallery
<code>PUBLIC_ROUTES</code>	<code>src/middleware.js</code>	Declared, never read	Remove
<code>get_resume_by_token</code> + <code>share_token</code>	<code>scripts/supabase.sql</code>	No route consumes it (public share links unimplemented)	Implement <code>/r/[token]</code> page or drop
<code>profiles</code> reads	SQL + app	Table/policies exist; app never queries it	Keep (future-proof) or note as dormant
<code>theme state</code>	<code>template/[id]/page.jsx</code>	Set but never affects rendering (passed to <code>downloadResumePDF</code> which ignores it)	Delete or implement theming
<code>keyAchievements</code> field	<code>templates/classic.json</code>	Never rendered by any template	Remove from seed
<code>makeEmpty().website</code> vs <code>websiteOrGithub</code>	<code>[id]/page.jsx</code>	Mismatched key name → <code>website</code> entries never display	Align names

11.2 Inconsistencies

- **Template data-shape drift:** `classic.json` uses `{role}, {school, year}`; `modern.json` uses `{name, tech}`; while `ResumeForm.defaultItem` uses `{title, company, institution, degree, fieldOfStudy}`. Renderers compensate with multi-shape normalizers (especially `ClassicTemplate`'s skill handling, and `ModernTemplate`'s summary array/string tolerance), but editing then saving a seeded template can silently drop fields the preview doesn't recognize.
- **Two autosave systems:** the form autosaves `resume_autosave_v_final` (debounced) and the store's `saveResume` writes `resume_saved` (manual only) — overlapping purposes with different keys; the `snapshot` id in `sessionStorage` makes cloud saves an *upsert-by-id* instead of a true upsert.
- **Error-handling granularity:** `resumeService` returns a mixed flag `{ok, auth, message}` where `auth:true` means "please sign in"; callers map that to distinct user messages (editor does this correctly). `handleShare`'s catch block lies ("Link copied!" even when clipboard failed).
- **Console logging left in production path:** `auth/callback/route.js` logs the full request URL (which at that moment contains the OAuth `code`) — a minor leak/inconvenience worth removing.
- **downloadPDF theme param:** the exporter's signature takes `theme` but ignores it; `ModernTemplate` / `ClassicTemplate` accept and ignore a `theme` prop too.
- **Duplicated json() helpers** across all API route files (4 copies) — a shared `lib/http.js` would DRY it.
- **Sitemap lists** `/dashboard` and `/analysis` — login-walled pages get indexed (SEO inconsistency).
- `npm run lint` is bare `eslint`; no `eslint .` scope, no CI pipeline, and **no tests at all** in the repo.

11.3 Security & robustness notes (mostly good)

- Service role key is server-side only; RLS covers all user data paths; webhooks HMAC-verified; payment server re-fetch (`payments.fetch`) double-checks capture; file size/type validated on both client and server.

- **Watch items:** (1) `.env.local` contains what look like real keys — it is gitignored, but keys should be rotated if the file was ever shared outside the machine; (2) the free quota check is a count-then-insert race (a fast client could burn >2 analyses in one month); (3) `getSupabaseAdmin` throws at call time if env missing — fine, but only the payment routes catch it; (4) HTML injected from `renderToString` into the export container is driven by user's own resume data — resume fields are rendered with JSX escaping (React escapes text), so XSS risk is low, but `dangerouslySetInnerHTML` is used in `layout.js` only for trusted static JSON-LD.
- **Performance:** the homepage lazy-loads every section below the fold (`next/dynamic`) and the PDF export caps pages at 60 — reasonable choices. The export's mandatory 800 ms font delay is a pragmatic hack.

11.4 What's well done

- Configuration-driven form (869 lines but zero per-field-UI duplication).
- Deep-clone discipline in the store prevents classic mutation bugs.
- The PDF pagination via `.pdf-section-start` markers is a clever, robust solution to an infamous jsPDF pain point.
- Resilient AI JSON parsing; quota counted only after success.
- Clean App-Router conventions throughout (layouts, metadata, error/loading boundaries, SEO files).

12. Glossary of Terms

Term	Meaning
ATS (Applicant Tracking System)	Software recruiters use to parse, filter, and score incoming resumes before human review. "Beating the ATS" = formatting/keyword choices that survive automated screening.
ATS score	AchivAI's 0–100 estimate of how well a resume would fare in ATS screening, produced by Gemini.
Route Handler	Next.js API endpoint: a <code>route.js</code> file exporting <code>GET</code> / <code>POST</code> /... functions.
RSC / Client Component	React Server Component (rendered on server, no JS bundle) vs. <code>"use client"</code> component (hydrated in the browser).
RLS (Row Level Security)	Postgres feature; each row's visibility is restricted per signed-in user via policies.
Service role key	Supabase full-privilege credential that bypasses RLS — never exposed to the browser.
SSR cookies / @supabase/ssr	Adapters that sync Supabase auth tokens through HTTP cookies so server code sees the same session as the browser.
HMAC signature	Keyed hash used to prove a message came from Razorpay (order/verify/webhook).
Webhook	Server-to-server HTTP callback Razorpay fires automatically on payment lifecycle events.
Magic link	Passwordless sign-in: Supabase emails a link; <code>/auth/callback</code> exchanges the enclosed code for a session.
Zustand	Tiny React state library (like Redux-lite) with a <code>create()</code> store + <code>useStore(selector)</code> hooks.
JSONB	PostgreSQL's binary JSON column type — the <code>resumes.data</code> field stores the whole resume document.
renderToString	React DOM server API that produces HTML from a component (used off-screen for PDF export).
html2canvas	Library that screenshots live DOM into a <code><canvas></code> ; here combined with jsPDF to build multi-page PDFs.
Quota / usage_events	Counting table that enforces "2 free analyses per calendar month".
Open Graph / JSON-LD	Social-share metadata and Schema.org structured data embedded for search-engine rich results.

End of report. Data gathered by reading every source file in the repository (63 application files + configs); no file was skimmed.